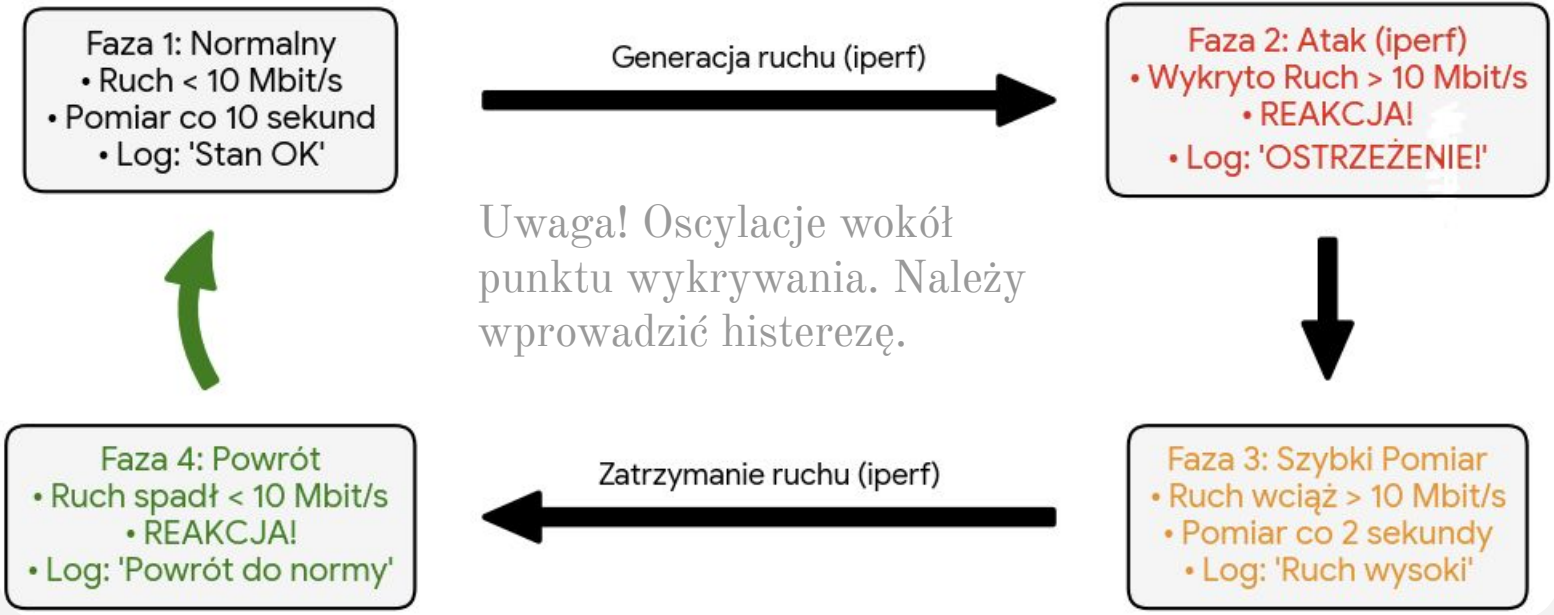
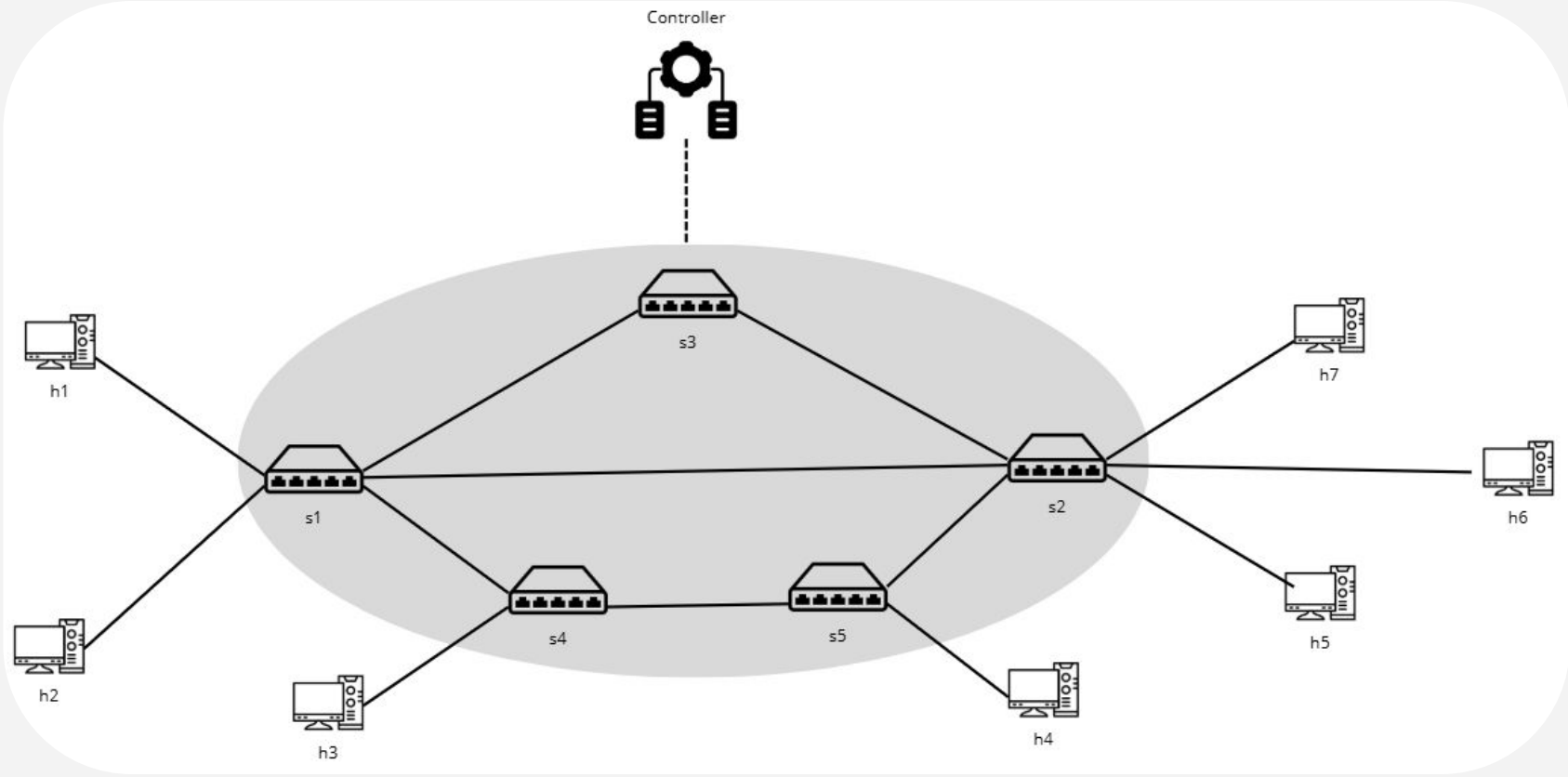
Dynamiczne pomiary ruchu w sieci z wykorzystaniem Ryu

Seratowicz Łukasz
Czajkowski Tomasz
Winiarski Filip
Jan Wojdat

Przebieg Scenariusza (Logika Sterownika Ryu)



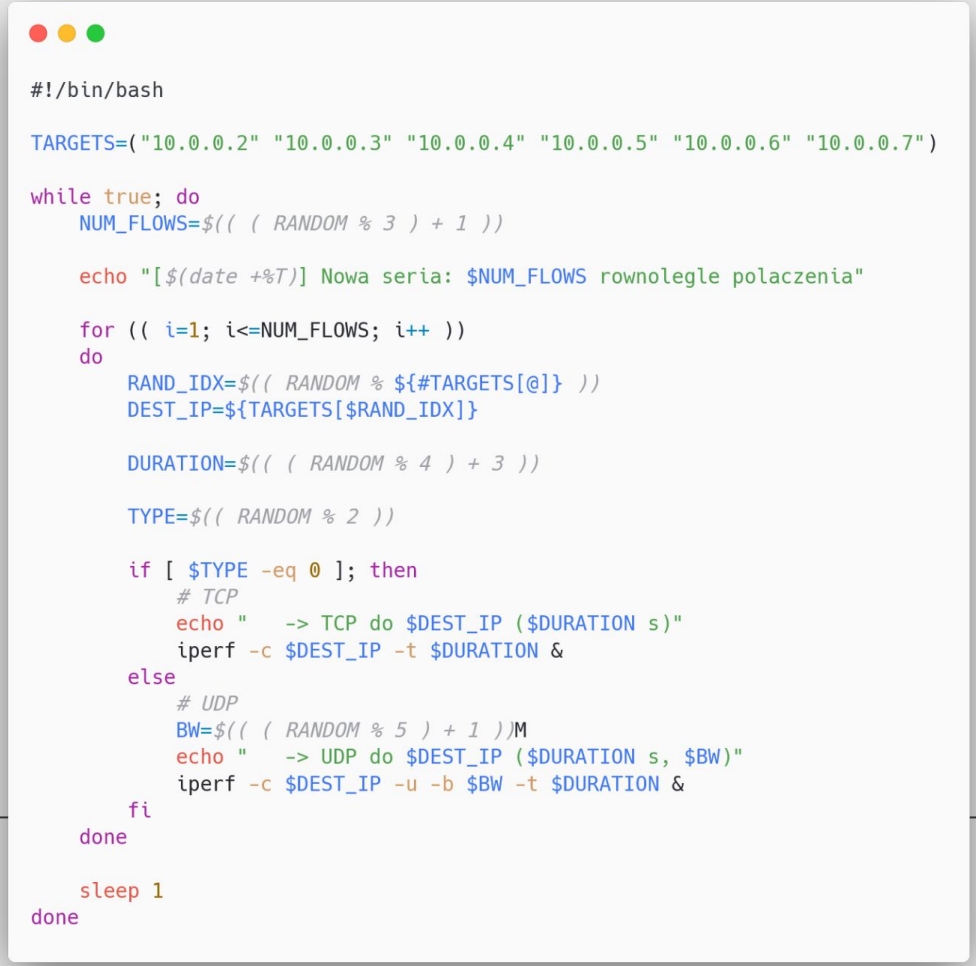
Topologia



Wykorzystane Technologie: Mininet + Ryu + Iperf

Mininet

iPerf



```bash
#!/bin/bash

TARGETS=("10.0.0.2" "10.0.0.3" "10.0.0.4" "10.0.0.5" "10.0.0.6" "10.0.0.7")

while true; do
    NUM_FLOWS=$(( ( RANDOM % 3 ) + 1 ))

    echo "[$(date +%T)] Nowa seria: $NUM_FLOWS rownolegle polaczenia"

    for (( i=1; i<=NUM_FLOWS; i++ ))
    do
        RAND_IDX=$(( RANDOM % ${#TARGETS[@]} ))
        DEST_IP=${TARGETS[$RAND_IDX]}

        DURATION=$(( ( RANDOM % 4 ) + 3 ))

        TYPE=$(( RANDOM % 2 ))

        if [ $TYPE -eq 0 ]; then
            # TCP
            echo "   -> TCP do $DEST_IP ($DURATION s)"
            iperf -c $DEST_IP -t $DURATION &
        else
            # UDP
            BW=$(( ( RANDOM % 5 ) + 1 ))M
            echo "   -> UDP do $DEST_IP ($DURATION s, $BW)"
            iperf -c $DEST_IP -u -b $BW -t $DURATION &
        fi
    done

    sleep 1
done
```

Generowanie Ruchu

skrypt w bash

Skrypt

```python
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import MAIN_DISPATCHER, CONFIG_DISPATCHER, se
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet
from ryu.app import simple_switch_stp_13
from ryu.lib import hub

import time


class ProjectController(simple_switch_stp_13.SimpleSwitch13):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(ProjectController, self).__init__(*args, **kwargs)
        self.datapaths = {}
        self.high_monitor_interval = 2
        self.low_monitor_interval = 10
        self.high_bw_threshold = 1000000
        self.low_bw_threshold = 800000
        self.flow_history = {}
        self.dp_next_poll = {}
        self.monitor_thread = hub.spawn(self._monitor)

    @set_ev_cls(ofp_event.EventOFPStateChange,
                [MAIN_DISPATCHER, CONFIG_DISPATCHER])
    def _state_change_handler(self, ev):
        datapath = ev.datapath
        if ev.state == MAIN_DISPATCHER:
            self.datapaths[datapath.id] = datapath
        else:
            self.datapaths.pop(datapath.id, None)
```

```python
    def _monitor(self):
        while True:
            now = time.time()
            next_query = self.low_monitor_interval
            for dp in self.datapaths.values():
                next_poll = self.dp_next_poll.get(dp.id, 0)
                if now >= next_poll:
                    self._request_flow_stats(dp)
                    next_query = 0
                else:
                    next_query = min(next_query, next_poll - now)
            if next_query != 0:
                hub.sleep(next_query)

    def _request_flow_stats(self, datapath):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        req = parser.OFPFlowStatsRequest(
            datapath=datapath,
            table_id=ofproto.OFPTT_ALL,
            out_port=ofproto.OFPP_ANY,
            out_group=ofproto.OFPG_ANY
        )
        datapath.send_msg(req)

    @set_ev_cls(ofp_event.EventOFPFlowStatsReply, MAIN_DISPATCHER)
    def _flow_stats_reply_handler(self, ev):
        now = time.time()
        dpid = ev.msg.datapath.id

        for stat in ev.msg.body:
            if stat.priority == 0:
                continue

            match_str = self._match_to_str(stat.match)
            key = (dpid, match_str)

            entry = self.flow_history.get(key)
            if entry is None:
                entry = {
                    'last_bytes': stat.byte_count,
                    'last_time': now,
                    'interval': self.low_monitor_interval,
                    'last_tp': 0.0,
                    'high_state': False
                }

                self.flow_history[key] = entry
                self.dp_next_poll[dpid] = now + self.low_monitor_interval
                continue
```

```python
            last_bytes = entry['last_bytes']
            last_time = entry['last_time']
            high_state = entry['high_state']
            state_str = "HIGH " if high_state else "LOW  "
            throughput_bps = 0
            if last_bytes is not None:
                delta_bytes = stat.byte_count - last_bytes
                delta_time = now - last_time
                if delta_time > 0:
                    throughput_bps = (delta_bytes * 8) / delta_time
            if throughput_bps >= self.high_bw_threshold and not high_state:
                entry['interval'] = self.high_monitor_interval
                entry['high_state'] = True
                state_str = "HIGH^"
            elif throughput_bps <= self.low_bw_threshold and high_state:
                entry['interval'] = self.low_monitor_interval
                entry['high_state'] = False
                state_str = "LOW v"

            if last_bytes is not None:
                self.logger.info(
                        "[THROUGHPUT] STATE=%s DPID=%016x Flow={%s} %.2f bps",
                        state_str, dpid, match_str, throughput_bps
                    )


            entry['last_bytes'] = stat.byte_count
            entry['last_time'] = now
            entry['last_tp'] = throughput_bps
            self.dp_next_poll[dpid] = now + entry['interval']

    def _match_to_str(self, match):
        return ",".join("{}={}".format(k,v) for k, v in sorted(match.items()))
```

Demo

Dziękujemy za uwagę!

Seratowicz Łukasz
Czajkowski Tomasz
Winiarski Filip
Jan Wojdat